

Week 1: MLFQ Scheduler Design Document

Suffiyan Asghar Ali (29182) Saad Thaplawala (27172)

November 19, 2025

1 Introduction

This document summarizes our Week 1 work for the MLFQ Scheduler project. The goal this week is not to build the full scheduler, but to set the foundation: understand xv6's current scheduling model, implement a small debugging syscall, and design how the MLFQ structure will look in later weeks.

2 Current xv6 Scheduling (Short Summary)

xv6's scheduler is very simple. It loops over the global process table and runs the first RUNNABLE process it finds. There is:

- No prioritization
- No queue structure
- No history of CPU usage
- No protection against starvation

This understanding guides the design of the MLFQ scheduler.

3 MLFQ Design (Week 1 Plan)

The MLFQ we will implement has four levels of priority. High-priority processes get short quanta; lower ones get longer quanta.

3.1 Queue Levels

- Q0 – Highest priority
- Q1
- Q2
- Q3 – Lowest priority

3.2 Time Quanta

Queue	Quantum (ticks)
Q0	1
Q1	2
Q2	4
Q3	8

3.3 Rules

- New processes always start in Q0.
- Scheduler always picks from the highest non-empty queue.
- If a process uses its full quantum, it moves down one level.
- If it voluntarily yields or waits on I/O, it stays in the same queue.
- A global boost will move all processes back to Q0 to prevent starvation.

4 Data Structures (Scaffolding Only)

Here is the design for queue scaffolding. We added placeholders for the queue system:

```
struct mlfq_queue {
    struct proc *procs[NPROC];
    int head;
    int tail;
};

struct mlfq_queue queues[4];
```

These will be used later when we implement enqueue, dequeue, and scheduling logic.

5 System Call: getprocinfo

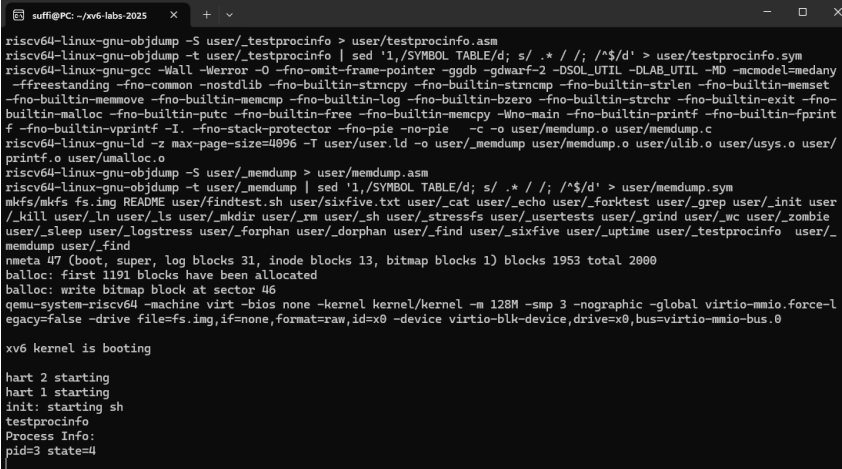
We added a simple syscall to inspect a process's basic information. This helps with debugging as we build the scheduler.

5.1 Prototype

```
struct procinfo {
    int pid;
    int state;
};

int getprocinfo(int pid, struct procinfo *info);
```

6 Appendix: Test Output Screenshot



```
suffi@PC: ~/xv6-labs-2025
riscv64-linux-gnu-objdump -S user/testprocinfo > user/testprocinfo.asm
riscv64-linux-gnu-objdump -t user/_testprocinfo | sed '1,/SYMBOL TABLE/d; s/ .* / /; /'$d' > user/testprocinfo.sym
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -DSOL_UTIL -DLAB_UTIL -MD -mcmmodel=medany
-ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset
-fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-
builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprint
f -fno-builtin-vprintf -fno-builtin-protect -fno-pie -no-pie -c -o user/memdump.o user/memdump.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_memdump user/memdump.o user/_lib.o user/_sys.o user/_
printf.o user/_umalloc.o
riscv64-linux-gnu-objdump -S user/_memdump > user/_memdump.asm
riscv64-linux-gnu-objdump -t user/_memdump | sed '1,/SYMBOL TABLE/d; s/ .* / /; /'$d' > user/_memdump.sym
mkfs/mkfs fs.img README user/findtest.sh user/_sixfive.txt user/_cat user/_echo user/_forktest user/_grep user/_init user
/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie
user/_sleep user/_logstress user/_forphan user/_dorphan user/_find user/_sixfive user/_uptime user/_testprocinfo user/_
memdump user/_find
mmeta 47 (boot: super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
ballocc: first 1191 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-l
egacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
testprocinfo
Process Info:
pid=3 state=4
```